Course: M.Sc. (IT) – Semester II

Subject: Artificial Intelligence & Machine Learning

Course Code: 250801202

Chapter 2

Game Playing

Game Playing in Artificial Intelligence

- Game playing is one of the **oldest and most important areas** of Artificial Intelligence.
- A game can be treated as a **search problem**, where AI searches for the **best possible move**.
- Many AI techniques were first developed using games like:
  - ✓ Chess
  - ✓ Tic-Tac-Toe
  - ✓ Checkers

Why study Game Playing?

- Helps us understand **decision making**
- Teaches how AI handles **competition and strategy**
- Used in **real-world problems** like planning and optimization

FCAIT MSc(IT) Programme

Game Playing in Artificial Intelligence

Components of a Game

- **Initial State**
  - Shows the **current board position**
  - Indicates **which player will move next**
- **Set of Operators**
  - All **legal moves** allowed in the game
  - Example: In chess, moving a pawn or rook legally
- **Terminal Test**
  - Checks whether the game is **finished**
  - Such a state is called a **terminal state**
- **Utility Function**
  - Gives a **numerical result** of the game
  - Example:
    - ✓ Win = +1
    - ✓ Loss = −1
    - ✓ Draw = 0

FCAIT MSc(IT) Programme

MINIMAX PROCEDURE

- Most games are **two-player games** or can be viewed as games played between two players.

- For example, in **chess**, one player plays with **white pieces** and the other with **black pieces**. At every turn, a player can make only **one legal move**.

- In the Minimax procedure, the two players are called **MAX player** and **MIN player**.
  - ✓ The **MAX player** is the player who tries to **win the game** by getting the **highest possible score (utility value)**.
  - ✓ The **MIN player** is the opponent who tries to **reduce MAX's score** and get the **lowest possible utility value**.

FCAIT MSc(IT) Programme

MINIMAX PROCEDURE

- The main goal of the Minimax procedure is to help both players **play optimally**.
- The **MAX player** always tries to reach a terminal state with the **maximum utility**, while the **MIN player** tries to reach a terminal state with the **minimum utility**.
- The game ends when a **terminal state** is reached.

FCAIT MSc(IT) Programme

Real-Life Analogy for MAX and MIN

Analogy: Student vs Examiner

- **MAX player = Student**
  - Wants **maximum marks**
  - Chooses answers that give the **highest possible score**
- **MIN player = Examiner**
  - Wants to **cut marks**
  - Looks for mistakes to give the **lowest possible marks**
- The student (MAX) prepares answers assuming the examiner (MIN) will be **strict**.
  So, the student chooses the **best and safest answer** to still score well.

This is exactly how Minimax works:

- MAX assumes MIN will act in the **worst possible way**
- MAX selects the move that gives the **best guaranteed outcome**

FCAIT MSc(IT) Programme

In a Simple way

Goal of the Minimax Procedure

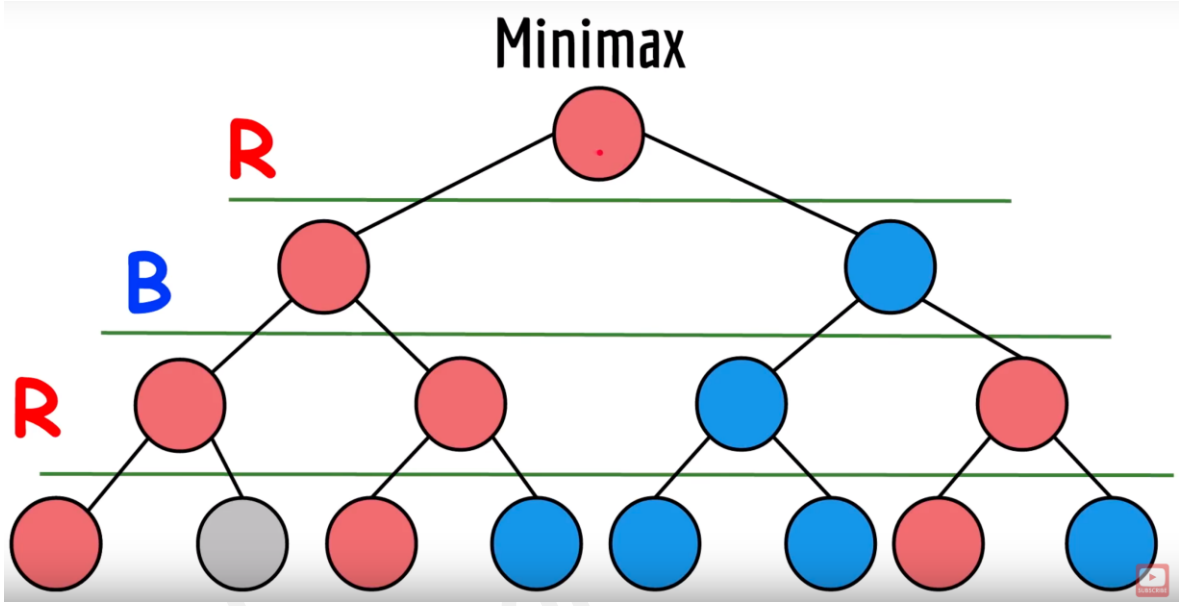
- Both players play **optimally**
- MAX tries to reach a terminal state with:
  Maximum utility value
- MIN tries to reach a terminal state with:
  Minimum utility value

Game ends when

- A **terminal state** is reached
- No further moves are possible
- *Student memory line:*
  "MAX hopes, MIN blocks."

FCAIT MSc(IT) Programme

MINIMAX PROCEDURE



Understanding a Minimax Game Tree

Root Node

- Represents current game position
- Always starts with **MAX**

Levels in Tree

- MAX level → chooses **maximum**
- MIN level → chooses **minimum**

Leaf Nodes

- Show **final outcomes**
- Have fixed numeric values

MINIMAX PROCEDURE

- **How it works:**
- The game starts from the **root node (top)**, which belongs to the **MAX player**.
- MAX makes a move → then it becomes **MIN's turn** → then again MAX's turn, and so on.
- At the **lowest level**, each node has a fixed value (win, loss, or score).
- These values are **passed upward**:
  - **MIN nodes choose the minimum value** from their children (to reduce MAX's gain).
  - **MAX nodes choose the maximum value** from their children (to increase their gain).
- Finally, the value at the **root node** tells MAX the **best possible move**.
- **Key idea:**
  MAX tries to **maximize profit**, MIN tries to **minimize loss**.

FCAIT MSc(IT) Programme

MINIMAX PROCEDURE

- Image shows a **numerical example of the Minimax procedure**.

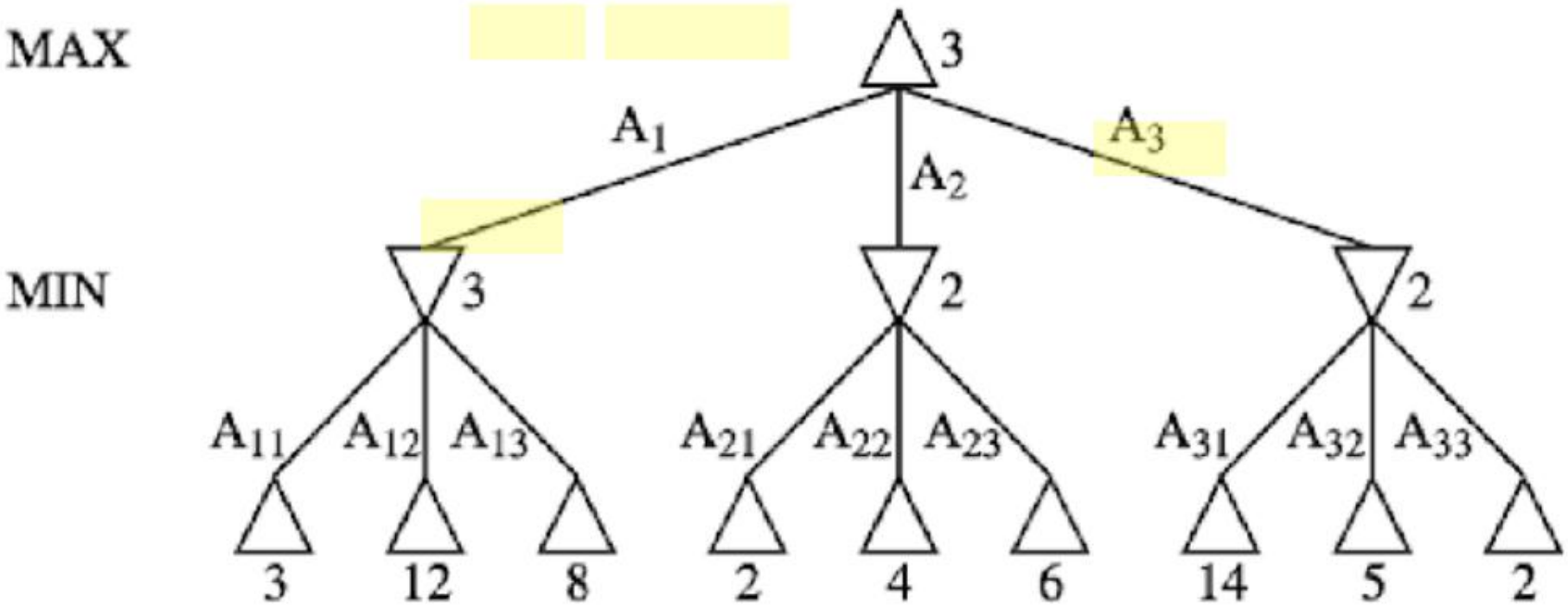


Figure 3.1 Two-player game: revisited.

MINIMAX PROCEDURE

Structure:

- Top node = MAX player
- Second level = MIN player
- Bottom level = terminal states with numeric values

Step-by-step:

- Start from the **leaf nodes** (values like 3, 12, 8, 2, 4, 6, etc.).
- Each **MIN node selects the smallest value** among its children:
  - Example: From (3, 12, 8), MIN selects **3**.
- These selected values move up to the **MAX node**.
- The **MAX node selects the highest value** among the values returned by MIN nodes.
- The final selected value tells MAX which action (**A1, A2, or A3**) is the **best move**.

Conclusion:

MAX assumes MIN will always play **against him**, so MAX chooses the safest and best option.

FCAIT MSc(IT) Programme

MINIMAX PROCEDURE

- The Minimax procedure can be explained using games like **Tic-Tac-Toe**, where the aim is to find the **best move for the MAX player**.
- In Minimax, a **heuristic (evaluation) function** is used to estimate how good a game position is:
- **e(p) = (Number of rows, columns, and diagonals open to MAX player) – (Number of rows, columns, and diagonals open to MIN player)**
- The algorithm generates all possible **next moves** up to a certain level and then **passes the values from the bottom (leaf nodes) to the top**.
- At a **MAX level**, the move with the **highest value** is chosen.
- At a **MIN level**, the move with the **lowest value** is chosen.
- Finally, the **MAX player selects the move that gives the best possible outcome**.

FCAIT MSc(IT) Programme

Which Formula to Apply in Minimax?

Why do we need a formula?

- In many games, we **cannot reach the final win/loss state**
- So, we **estimate** how good a position is
- This estimation is done using an **Evaluation (Heuristic) Function**

Evaluation Function Formula

$$e(p) = (\textit{Open lines for MAX}) - (\textit{Open lines for MIN})$$

What does "Open line" mean?

- A row, column, or diagonal
- That is **not blocked by the opponent**
- And can still lead to a win

How to interpret the value?

e(p) > 0 → Position is **good for MAX**
e(p) < 0 → Position is **good for MIN**
e(p) = 0 → Position is **balanced**

FCAIT MSc(IT) Programme

MINIMAX PROCEDURE

Opportunities for MAX (X)

- Main diagonal (X, X, blank)
- Second row (blank, X, blank)
- First column (X, blank, blank)

  Total = 3

Opportunities for MIN (O)

- Third column (O, blank, blank)
- Third row (blank, O, blank)

  Total = 2

Evaluation Value :- e(p) = 3 – 2 = +1

Positive value → MAX is in a better position

Example :-

```
X |   | O
---+---+---
  | X |
---+---+---
  | O |
```

FCAIT MSc(IT) Programme

MINIMAX ALGORITHM

- Generate all possible game moves as a **tree**.
- Assign **utility values** to final (leaf) states.
- Move **bottom to top**:
  - ✓ **MAX** chooses the **maximum value**
  - ✓ **MIN** chooses the **minimum value**
- Continue until the **root node** is reached.
- MAX selects the move with the **best guaranteed outcome**, assuming MIN plays optimally

FCAIT MSc(IT) Programme

MINIMAX PROCEDURE

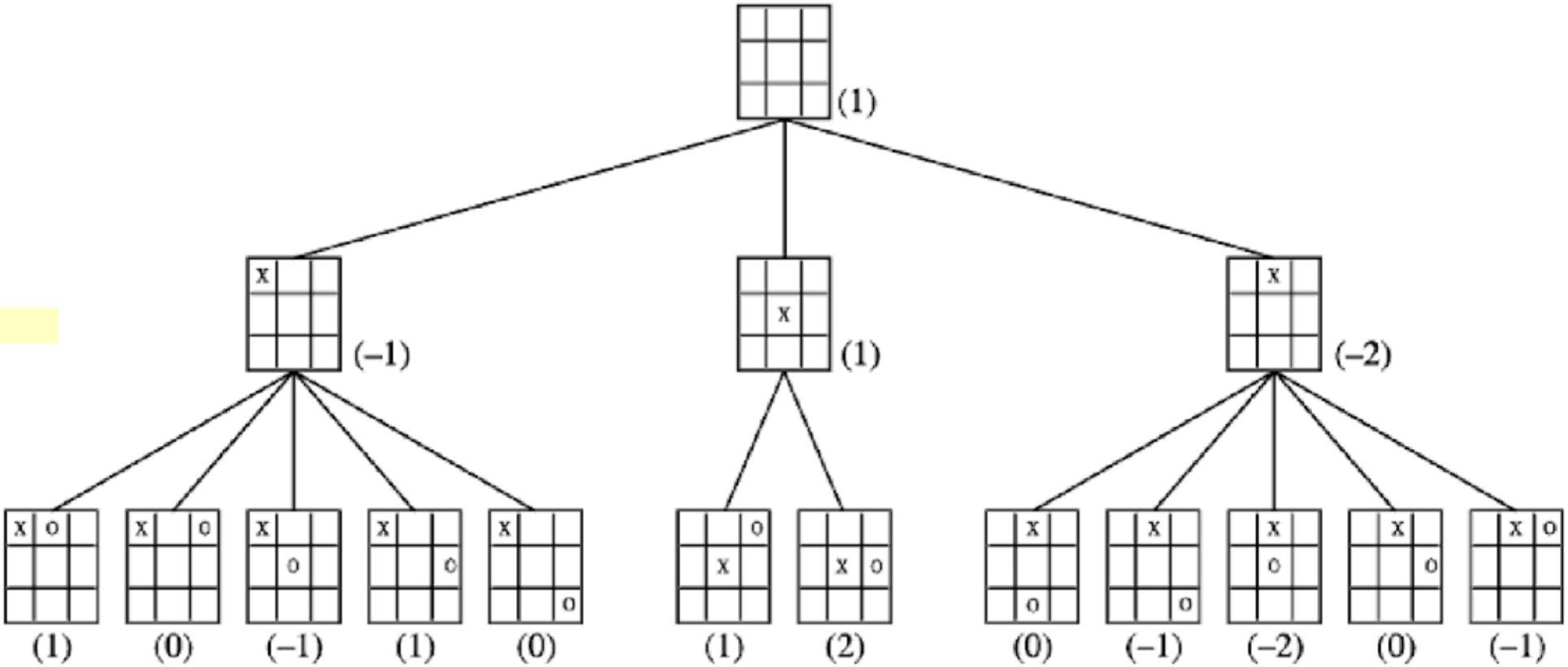


Figure 3.3 Two step graph of MINIMAX procedure.
X - MAX Player, O – MIN Player

MINIMAX PROCEDURE

Minimax Solution (Two-Step Graph)

Leaf nodes are evaluated using

e(p) = (Opportunities for MAX) − (Opportunities for MIN)

MIN player selects the **minimum value** from its child nodes:

- Left branch → **−1**
- Middle branch → **+1**
- Right branch → **−2**

MAX player selects the **maximum value** among MIN outcomes:

max(−1, +1, −2) = +1

Final Minimax Value = +1

Positive value ⇒ MAX (X) is in a better position

FCAIT MSc(IT) Programme

MINIMAX ALGORITHM

```
function MINIMAX(node, depth, maximizingPlayer)

    // Base condition: stop recursion
    if depth == 0 OR node is a terminal node then
        return evaluation(node)          // return heuristic / utility value
    end if

    // Case 1: MAX player's turn
    if maximizingPlayer == true then
        maxValue = -∞                    // initialize to lowest possible value

        for each child of node do
            value = MINIMAX(child, depth - 1, false)
            maxValue = max(maxValue, value)   // choose maximum value
        end for

        return maxValue                  // best value for MAX player

    // Case 2: MIN player's turn
    else
        minValue = +∞                    // initialize to highest possible value

        for each child of node do
            value = MINIMAX(child, depth - 1, true)
            minValue = min(minValue, value)   // choose minimum value
        end for

        return minValue                  // best value for MIN player
    end if

end function
```

MINIMAX ALGORITHM

MINIMAX FUNCTION

If **depth = 0** or **game is over** → return evaluation value.

If it is **MAX's turn**:

Start with **−∞**
Check all children
Choose the **maximum value**

If it is **MIN's turn**:

Start with **+∞**
Check all children
Choose the **minimum value**

Return the selected value.

FCAIT MSc(IT) Programme

MINIMAX PROCEDURE

- **MINIMAX ALGORITHM**
  This algorithm helps a computer decide the **best move** in a two-player game.

Step-by-step logic:

The function **minimax(node, depth, maximizingPlayer)** is called.

If:

- ✓ the required **depth is reached**, or
- ✓ the node is a **terminal state**
  → return the **evaluation value** of that node.

FCAIT MSc(IT) Programme

MINIMAX PROCEDURE

If the player is MAX (maximizingPlayer = true):

- Start with the smallest possible value (**−∞**).
- Check all child nodes one by one.
- Call minimax recursively for each child.
- Select the **maximum value** among them.
- Return this maximum value.
- MAX always chooses the **best possible score**.

If the player is MIN (maximizingPlayer = false):

- Start with the largest possible value (**+∞**).
- Check all child nodes.
- Call minimax recursively for each child.
- Select the **minimum value** among them.
- Return this minimum value.
- MIN always tries to **reduce MAX's score**.

FCAIT MSc(IT) Programme

Limitations of MINIMAX PROCEDURE

1. Works well only for small games

Minimax is useful when a game has **few possible moves**.

If a game has **too many choices**, Minimax becomes **slow and impractical**.

2. Time Complexity of Minimax

Time complexity is **$O(m^n)$**

m = average number of moves at each step

n = number of plies (levels) searched

Example: Chess

About **35 possible moves** at each step, Around **100 moves** in a game

Total possibilities = **35^{100}** (very huge!)

3. Need for Cut-off

Because search space is very large:

Computer **stops search early** (cut-off)

Uses **evaluation function** instead of final result

Cut-off is required due to **time and memory limits**

FCAIT MSc(IT) Programme

Problems with Evaluation Function

1. Incorrect Decision after Cut-off

If search stops early:

Important future moves (like captures) may be **missed**

Evaluation function may give a **wrong value**

2. Horizon Problem

Computer searches only **few levels ahead** (e.g., 5 plies)

A bad move exists **after that depth**

Since computer cannot see that far:

It makes a **wrong decision**

This problem is called the **Horizon Problem**

(*Error is hidden beyond the search depth*)

FCAIT MSc(IT) Programme

Improvement: Quiescent Search

Why needed?

To reduce wrong decisions near cut-off

Key Idea

- Apply evaluation function **only on stable positions**
- Avoid positions with:
  - ✓ Sudden captures
  - ✓ Big score changes
- Such stable positions are called **Quiescent Positions**

Quiescent Search

- Extends search **only when position is unstable**
- Stops when position becomes **calm and safe**
- Gives **more accurate evaluation**

FCAIT MSc(IT) Programme

ALPHA–BETA PRUNING

- Minimax checks **all possible branches** of the game tree.
- Many of these branches **do not affect the final decision**.
- Searching all branches makes Minimax **slow and inefficient**.
- To improve Minimax, we use **Alpha–Beta Pruning**.

Why Alpha–Beta Pruning?

- Minimax wastes time exploring **unnecessary branches**.
- Alpha–Beta Pruning **cuts off** such branches.
- These branches are **ignored without being examined**.
- Result:
  - ✓ Faster search
  - ✓ Same result as Minimax
  - ✓ Fewer nodes examined

FCAIT MSc(IT) Programme

ALPHA–BETA PRUNING

Basic Idea of Alpha–Beta Pruning

- Alpha–Beta Pruning is a **smart version of Minimax**.
- It searches the game tree in a **depth-first manner**.
- It eliminates branches that **cannot influence the final answer**.

Important point:

- Alpha–Beta Pruning returns the **same optimal move as Minimax**, but faster.

FCAIT MSc(IT) Programme

ALPHA–BETA PRUNING

What is Pruning?

Pruning means:

Stopping the search of a branch

Because it is **not useful**

Pruned branches:

Are **never evaluated**

Do **not affect the final decision**

This saves **time and computation**.

FCAIT MSc(IT) Programme

Two Important Values in Alpha–Beta Pruning

During the search, two values are used:

1. Alpha (α)

Associated with **MAX nodes**

Represents the **best value MAX can guarantee so far**

2. Beta (β)

Associated with **MIN nodes**

Represents the **best value MIN can guarantee so far**

Meaning of Alpha and Beta

Alpha (α) → "MAX already has at least this much value"

Beta (β) → "MIN will allow at most this much value"

These values help decide **when to stop searching**.

FCAIT MSc(IT) Programme

Two Important Values in Alpha–Beta Pruning

Rule 1: Beta Cut-off (MIN Node)

- **Condition:** If **β ≤ α** at a **MIN node**
- **Meaning:**
- MIN has found a value **worse for MAX**
- MAX already has a **better option elsewhere**
- So this branch is **pruned (cut off)**.

Rule 2: Alpha Cut-off (MAX Node)

- **Condition:** If **α ≥ β** at a **MAX node**
- **Meaning:**
- MAX has found a value **better than what MIN allows**
- MIN will never choose this branch
- So this branch is **pruned**.

FCAIT MSc(IT) Programme

Time Complexity of Alpha–Beta Pruning

Let:

m = branching factor

n = depth of the tree

Best case complexity:

$$O\left(m^{n/2}\right)$$

This is **much faster** than Minimax (which is $O(m^n)$)

FCAIT MSc(IT) Programme

Important Points to Remember

- Alpha–Beta Pruning:
  Does **not change the final answer**
  Only reduces the number of nodes searched
- It is an **optimization of Minimax**
- Works best when:
  Good moves are examined **first**

FCAIT MSc(IT) Programme

ALPHA–BETA PRUNING

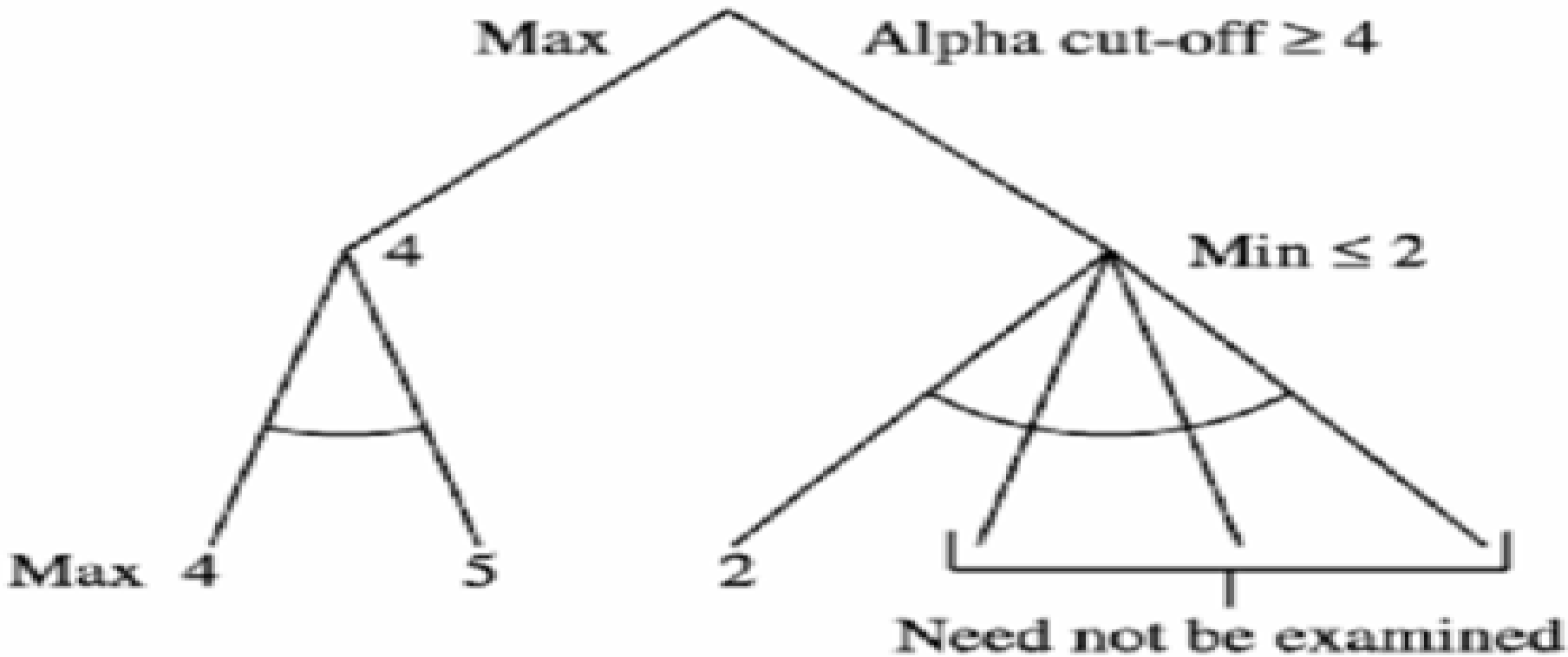


Figure 3.4 *a–b* cut-off example.

ALPHA–BETA PRUNING

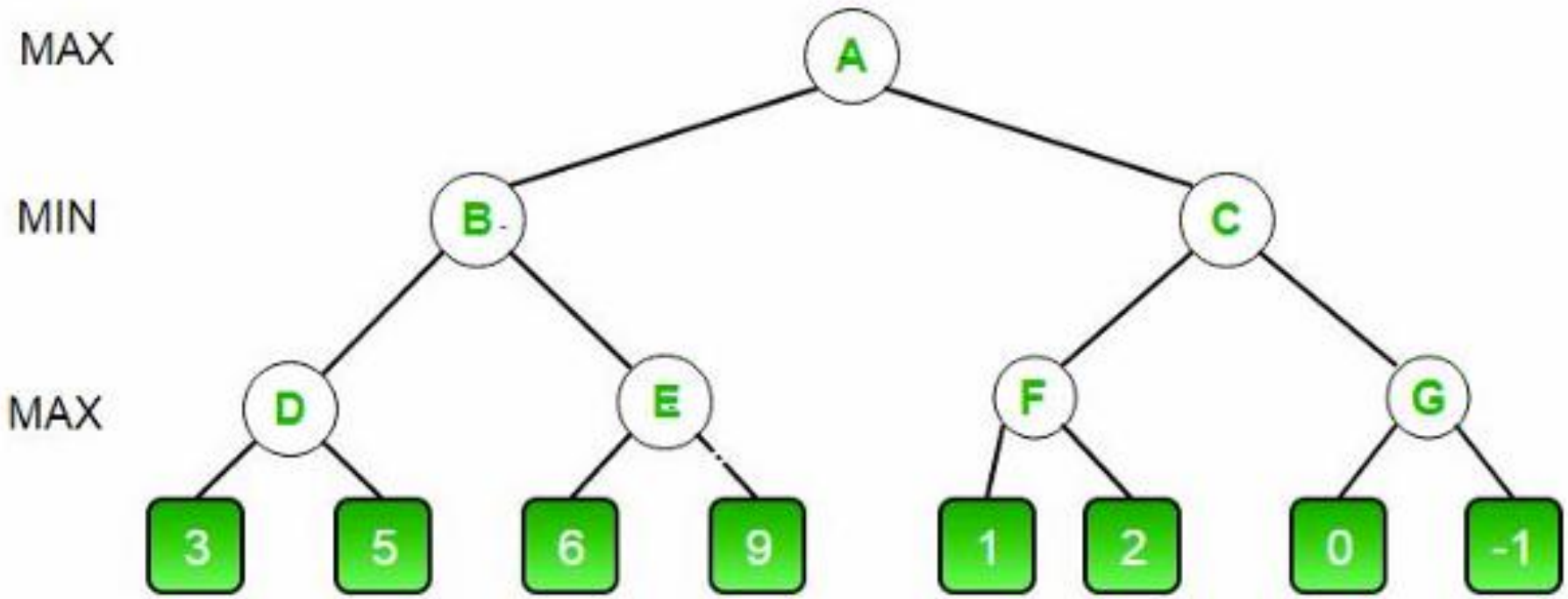
- The search starts from the **MAX node**.
- From the left branch, MAX gets value **4** → **α = 4**.
- Now MIN starts evaluating the right branch.
- MIN finds a value **2** → **β = 2**.
- Since **β ≤ α (2 ≤ 4)**:
  - MAX already has a better option
  - Remaining branches **need not be examined**
- This is called **Alpha cut-off**.

FCAIT MSc(IT) Programme

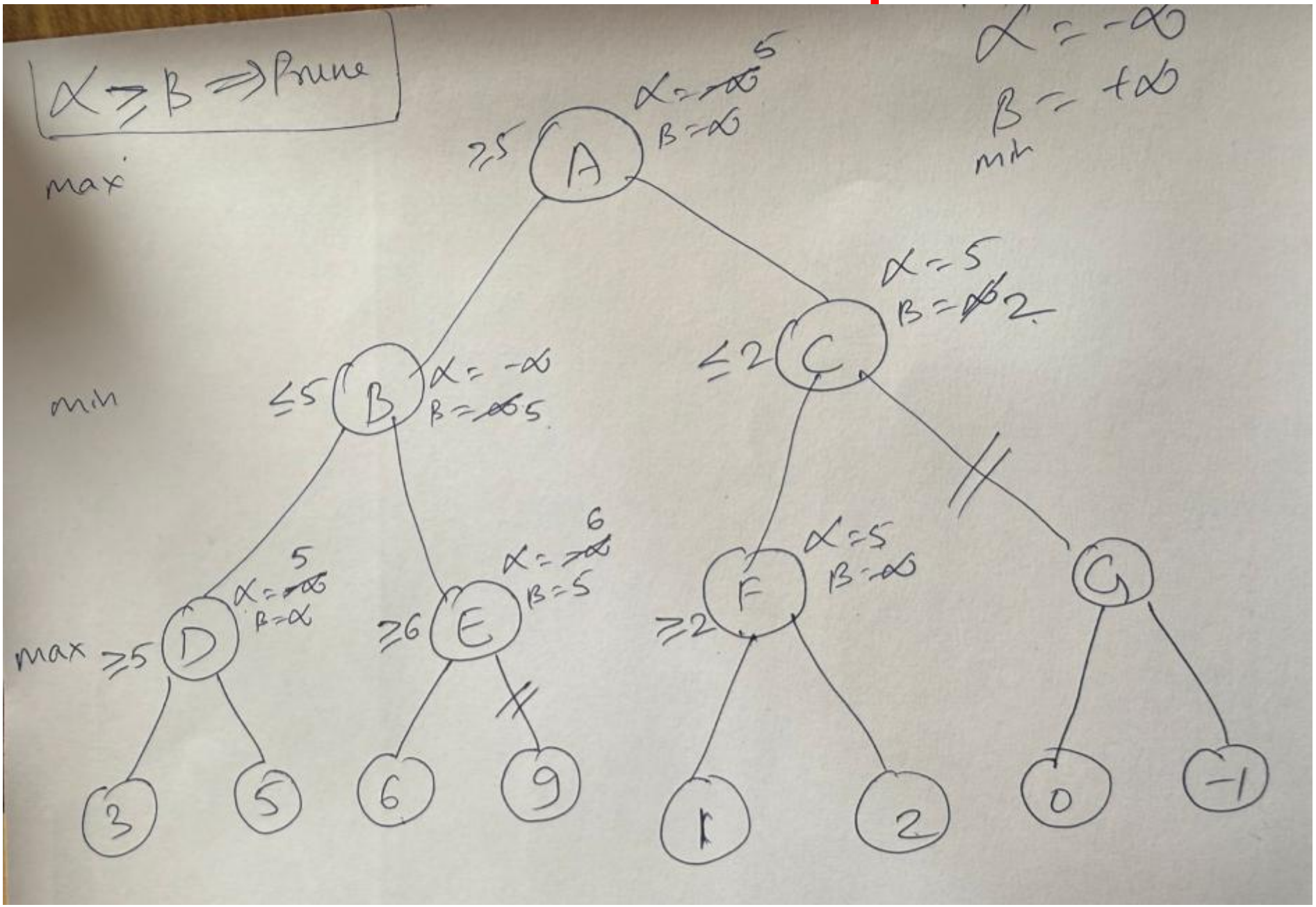
Conclusion

- Some branches are **pruned without evaluation**
- Final result remains **same as Minimax**
- Search becomes **faster**

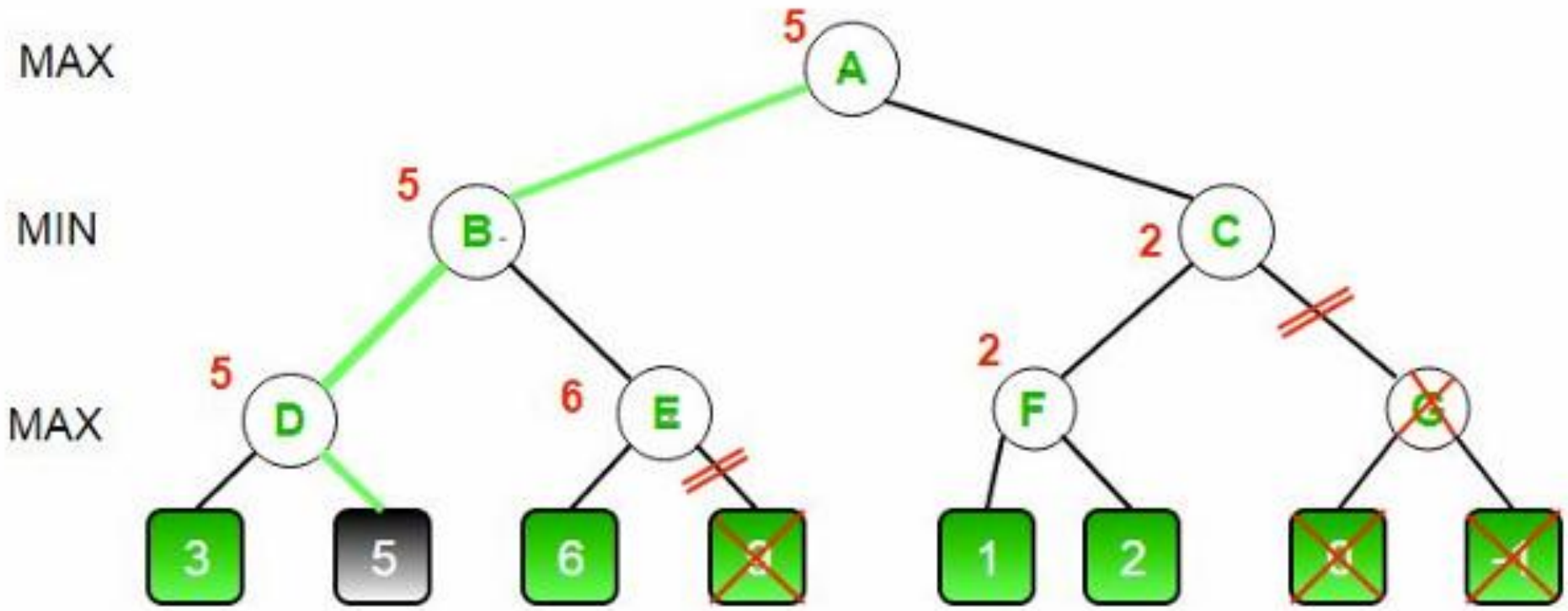
ALPHA–BETA PRUNING Example



ALPHA–BETA PRUNING Example



ALPHA–BETA PRUNING
Example Solution



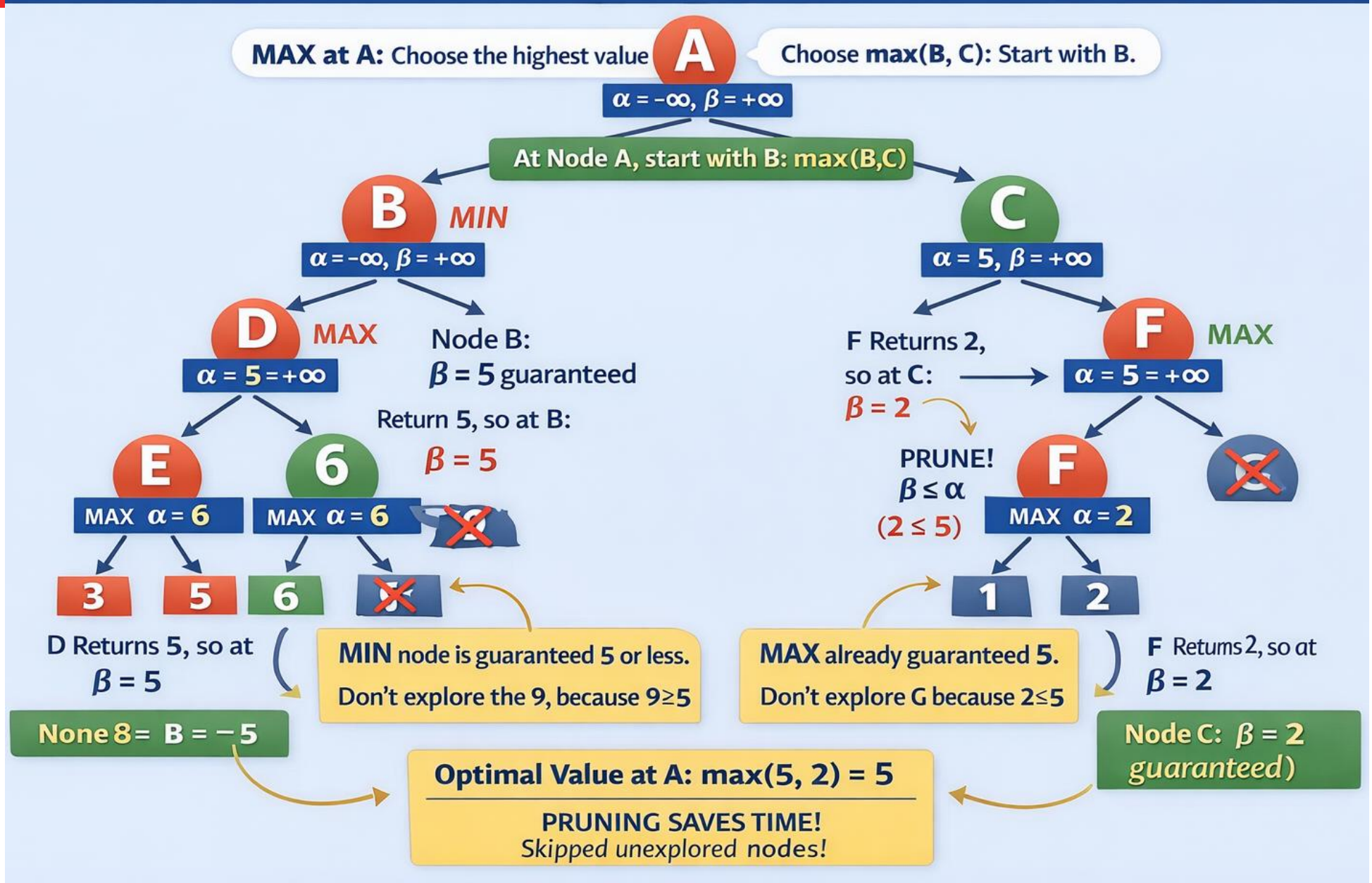
ALPHA–BETA PRUNING

- **A** is the **MAX** node (root).
- **B and C** are **MIN** nodes.
- **D, E, F, G** are **MAX** nodes.
- Leaf nodes show the **final values** of the game.

What this image represents

- This is the **full game tree before pruning**.
- All possible moves are shown.
- Alpha–Beta pruning will decide **which branches to examine** and **which to ignore**.

- This image is used to **apply Alpha–Beta rules step by step** in the next slide.

FCAIT MSc(IT) Programme

Alpha-Beta Pruning Explained